

Cyclistic Bike-Share Analysis (Python Documentation)

Step 1: Sample Dataset Creation (BigQuery)

This step creates a **2% sampled dataset** for efficient Python-based analysis.

CREATE OR REPLACE TABLE

```
`model-myth-487516-u3.cyclistic_analysis.cyclistic_python_sample`
```

AS

SELECT

```
ride_duration,  
distance_miles,  
member_casual,  
rideable_type,  
hour_of_day,  
day_of_week,  
day_type,  
season,  
trip_type,  
duration_bucket,  
time_segment
```

```
FROM `model-myth-487516-u3.cyclistic_analysis.cyclistic_2025_cleaned`
```

```
TABLESAMPLE SYSTEM (2 PERCENT);
```

Step 2: Environment Setup & Library Imports

This section initializes required libraries and defines global styling for consistent visualization.

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.patches as mpatches

import seaborn as sns

from scipy import stats

from scipy.stats import mannwhitneyu

import warnings

warnings.filterwarnings('ignore')


# — Global color palette (matches your Tableau dashboard) —
COLORS = {

    'member' : '#E6820E', # orange

    'casual' : '#6A73C1', # blue-purple

    'both'   : ['#6A73C1', '#E6820E'],

    'neutral' : '#888780'

}


# — Global plot style —
plt.rcParams.update({

    'figure.facecolor' : 'white',

    'axes.facecolor'   : 'white',

    'axes.spines.top'   : False,

    'axes.spines.right': False,

    'axes.labelsize'   : 12,

    'axes.titlesize'   : 14,

    'axes.titleweight' : 'bold',

    'xtick.labelsize'  : 10,

    'ytick.labelsize'  : 10,

    'legend.fontsize'  : 10,

    'font.family'      : 'sans-serif'

})


print("✔ Libraries imported successfully")
```

⬇ Step 3: Data Loading & Initial Validation

Load dataset and perform initial sanity checks.

— Upload cyclistic_python_sample.csv to Colab first —

File → Upload to session storage

```
df = pd.read_csv('/content/cyclistic_python_sample.csv')
```

Quick sanity check

```
print(f"Shape: {df.shape}")
```

```
print(f"Columns: {list(df.columns)}")
```

```
print(f"\nRider type split:")
```

```
print(df['member_casual'].value_counts())
```

```
print(f"\nNull values:")
```

```
print(df.isnull().sum())
```

📊 Step 4: Descriptive Statistics & Data Profiling

Provides statistical overview of ride behavior by user type.

— Section 1: Descriptive Statistics —

```
print("=" * 55)
```

```
print(" DESCRIPTIVE STATISTICS")
```

```
print("=" * 55)
```

Summary by rider type

```
summary = df.groupby('member_casual').agg(
```

```
    total_rides = ('ride_duration', 'count'),
```

```
    mean_duration = ('ride_duration', 'mean'),
```

```
    median_duration= ('ride_duration', 'median'),
```

```
    std_duration = ('ride_duration', 'std'),
```

```

min_duration = ('ride_duration', 'min'),
max_duration = ('ride_duration', 'max'),
mean_distance = ('distance_miles', 'mean'),
median_distance= ('distance_miles', 'median')
).round(2)

```

```

print(summary.T) # Transpose for readability

```

```

# Percentage split

```

```

total = len(df)

```

```

for rtype, count in df['member_casual'].value_counts().items():
    print(f"\n{rtype}: {count:}, rides ({count/total*100:.1f}%")

```

```

=====

```

DESCRIPTIVE STATISTICS

```

=====

```

member_casual	casual	member
total_rides	45557.00	82838.00
mean_duration	19.47	11.60
median_duration	11.00	8.00
std_duration	38.83	18.95
min_duration	1.00	1.00
max_duration	1428.00	1438.00
mean_distance	1.40	1.41
median_distance	1.04	1.02

member: 82,838 rides (64.5%)

casual: 45,557 rides (35.5%)

🔗 Step 5: Hypothesis Testing & Statistical Validation

This section validates behavioral differences using statistical tests.

```
# — Section 2: Hypothesis Testing —
```

```
print("=" * 55)
```

```
print(" HYPOTHESIS TESTING")
```

```
print("=" * 55)
```

```
member = df[df['member_casual'] == 'member']['ride_duration']
```

```
casual = df[df['member_casual'] == 'casual']['ride_duration']
```

```
# — Test 1: Welch's t-test (unequal variance) —
```

```
t_stat, p_value = stats.ttest_ind(member, casual, equal_var=False)
```

```
print("\n🚀 Test 1 — Welch's T-Test (Ride Duration)")
```

```
print(f" H0: No difference in mean ride duration")
```

```
print(f" H1: Casual riders have longer mean duration")
```

```
print(f" T-statistic : {t_stat:.4f}")
```

```
print(f" P-value : {p_value:.2e}")
```

```
print(f" Result : {'✅ Reject H0 — significant difference' if p_value < 0.05 else '❌ Fail to reject H0'}")
```

```
# — Test 2: Cohen's d (effect size) —
```

```
def cohen_d(a, b):
```

```
    pooled_std = np.sqrt((np.std(a, ddof=1)**2 + np.std(b, ddof=1)**2) / 2)
```

```
    return (np.mean(a) - np.mean(b)) / pooled_std
```

```
d = cohen_d(member, casual)
```

```
magnitude = 'small' if abs(d) < 0.5 else 'medium' if abs(d) < 0.8 else 'large'
```

```
print(f"\n🚀 Test 2 — Cohen's d (Effect Size)")
```

```

print(f" Cohen's d : {d:.4f}")
print(f" Magnitude : {magnitude} effect")
print(f" Interpretation: Members ride {abs(d):.2f} standard deviations")
print(f" shorter than casual riders on average")

# — Test 3: Mann-Whitney U (non-parametric, more robust) —
u_stat, p_mw = mannwhitneyu(member, casual, alternative='less')

print(f"\n🚀 Test 3 — Mann-Whitney U Test (Non-Parametric)")
print(f" U-statistic : {u_stat:.0f}")
print(f" P-value : {p_mw:.2e}")
print(f" Result : {'✅ Members significantly shorter rides' if p_mw < 0.05 else '❌ No significant difference'}")

# — Test 4: Distance t-test —
m_dist = df[df['member_casual'] == 'member']['distance_miles']
c_dist = df[df['member_casual'] == 'casual']['distance_miles']
t_dist, p_dist = stats.ttest_ind(m_dist, c_dist, equal_var=False)

print(f"\n🚀 Test 4 — Distance T-Test")
print(f" P-value : {p_dist:.4f}")
print(f" Result : {'Significant' if p_dist < 0.05 else 'Not significant'} difference in distance")
print(f" Note : Mean distance nearly identical")
print(f" Member: {m_dist.mean():.2f} mi | Casual: {c_dist.mean():.2f} mi")

=====

HYPOTHESIS TESTING

=====

```

🚀 Test 1 — Welch's T-Test (Ride Duration)

H0: No difference in mean ride duration

H1: Casual riders have longer mean duration

T-statistic : -40.7002

P-value : 0.00e+00

Result : ✓ Reject H0 — significant difference

★ Test 2 — Cohen's d (Effect Size)

Cohen's d : -0.2577

Magnitude : small effect

Interpretation: Members ride 0.26 standard deviations
shorter than casual riders on average

★ Test 3 — Mann-Whitney U Test (Non-Parametric)

U-statistic : 1459368167

P-value : 0.00e+00

Result : ✓ Members significantly shorter rides

★ Test 4 — Distance T-Test

P-value : 0.0306

Result : Significant difference in distance

Note : Mean distance nearly identical

Member: 1.41 mi | Casual: 1.40 mi

Step 6: Ride Duration Distribution Analysis

Visual comparison of ride duration patterns.

— Section 3: Ride Duration Distribution —

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
fig.suptitle('Ride Duration Analysis: Member vs Casual',  
            fontsize=15, fontweight='bold', y=1.02)
```

Plot 1 — Overlapping histogram with KDE

```

ax1 = axes[0]
for rtype, color in [('casual', COLORS['casual']),
                     ('member', COLORS['member'])]:
    data = df[df['member_casual'] == rtype]['ride_duration']
    ax1.hist(data, bins=50, alpha=0.45, color=color,
             label=rtype.capitalize(), density=True, range=(0,60))
    data.plot.kde(ax=ax1, color=color, linewidth=2.5,
                 xlim=(0,60))

ax1.axvline(casual.median(), color=COLORS['casual'],
            linestyle='--', alpha=0.8, linewidth=1.5,
            label=f'Casual median: {casual.median():.0f} min')
ax1.axvline(member.median(), color=COLORS['member'],
            linestyle='--', alpha=0.8, linewidth=1.5,
            label=f'Member median: {member.median():.0f} min')
ax1.set_title('Ride Duration Distribution (0–60 min)')
ax1.set_xlabel('Ride Duration (minutes)')
ax1.set_ylabel('Density')
ax1.legend()

```

Plot 2 — Boxplot

```

ax2 = axes[1]
bp = ax2.boxplot(
    [casual.clip(0,60), member.clip(0,60)],
    labels=['Casual', 'Member'],
    patch_artist=True,
    medianprops=dict(color='white', linewidth=2.5),
    whiskerprops=dict(linewidth=1.5),
    capprops=dict(linewidth=1.5),
    flierprops=dict(marker='o', markersize=2, alpha=0.3)
)

```



```

bp['boxes'][0].set_facecolor(COLORS['casual'])

bp['boxes'][1].set_facecolor(COLORS['member'])

ax2.set_title('Ride Duration Boxplot (capped 60 min)')
ax2.set_xlabel('Rider Type')
ax2.set_ylabel('Ride Duration (minutes)')

# Annotate means
for i, (rtype, color) in enumerate(
    [('casual', COLORS['casual']), ('member', COLORS['member'])], 1):
    mean_val = df[df['member_casual']==rtype]['ride_duration'].mean()
    ax2.annotate(f'Mean: {mean_val:.1f}m',
                 xy=(i, mean_val), xytext=(i+0.22, mean_val),
                 fontsize=9, color=color, fontweight='bold',
                 arrowprops=dict(arrowstyle='->', color=color,
                                lw=1.5))

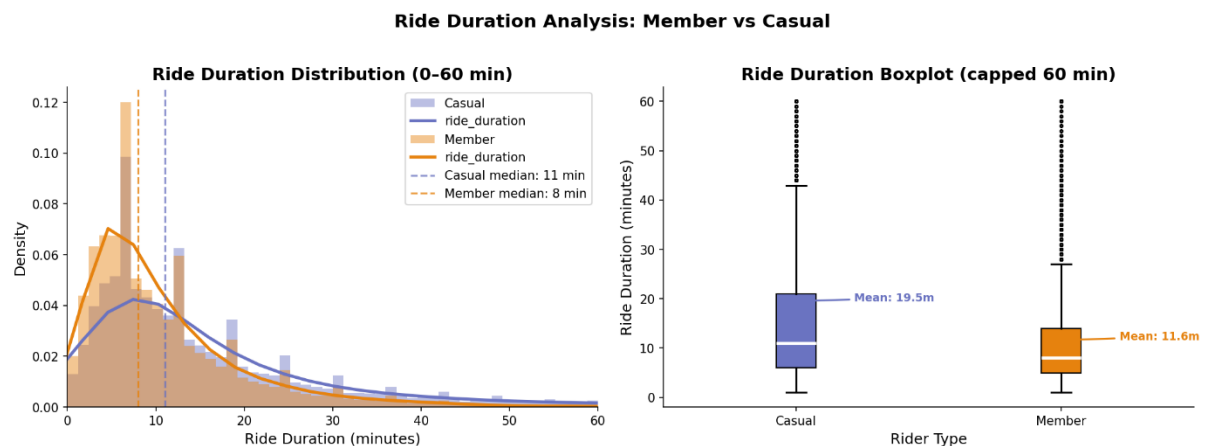
plt.tight_layout()

plt.savefig('01_ride_duration_distribution.png',
           dpi=150, bbox_inches='tight')

plt.show()

print("✓ Saved: 01_ride_duration_distribution.png")

```



Step 7: Duration Bucket Comparison

Analyzes ride distribution across predefined duration segments.

— Section 4: Duration Bucket Comparison —

```
bucket_order = ['0–10 min', '11–20 min', '21–40 min', '40+ min']

bucket_data = df.groupby(['member_casual', 'duration_bucket'])\
    .size().reset_index(name='rides')

bucket_data['pct'] = bucket_data.groupby('member_casual')['rides']\
    .transform(lambda x: x / x.sum() * 100)

fig, ax = plt.subplots(figsize=(11, 6))

x = np.arange(len(bucket_order))
width = 0.35

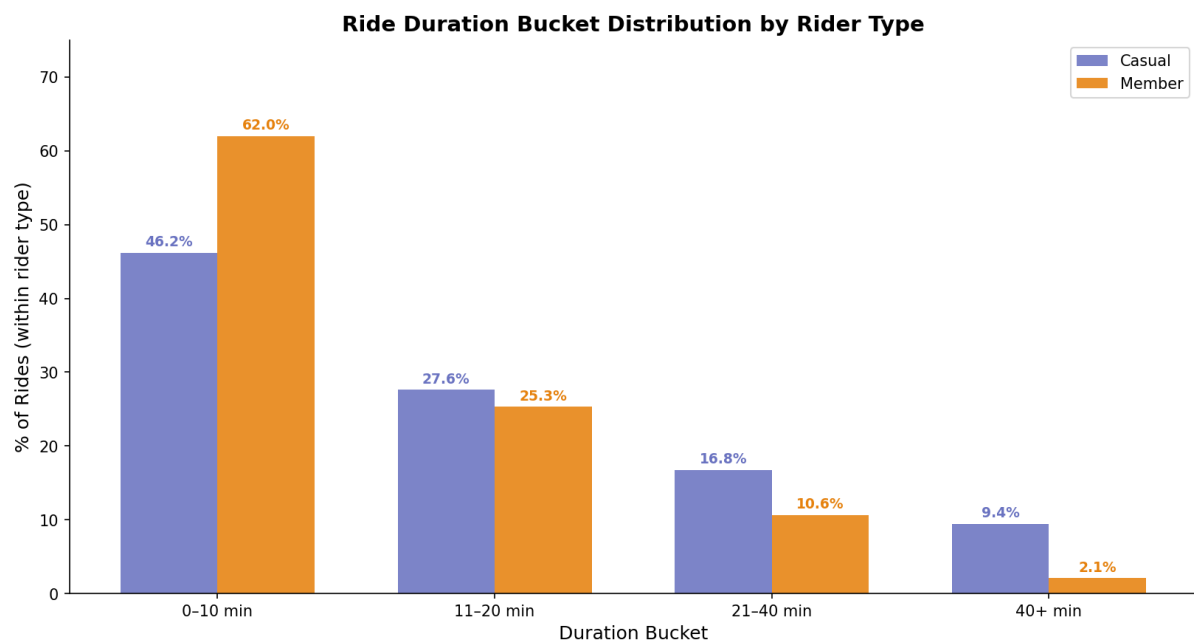
for i, (rtype, color) in enumerate(
    [('casual', COLORS['casual']), ('member', COLORS['member'])]):
    vals = []
    for b in bucket_order:
        row = bucket_data[
            (bucket_data['member_casual'] == rtype) &
            (bucket_data['duration_bucket'] == b)
        ]
        vals.append(row['pct'].values[0] if len(row) > 0 else 0)

    bars = ax.bar(x + (i - 0.5) * width, vals, width,
        label=rtype.capitalize(), color=color, alpha=0.88)

    for bar, val in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
```

```
f'{val:.1f}%', ha='center', va='bottom',  
fontsize=9, fontweight='bold', color=color)
```

```
ax.set_xticks(x)  
ax.set_xticklabels(bucket_order)  
ax.set_title('Ride Duration Bucket Distribution by Rider Type')  
ax.set_xlabel('Duration Bucket')  
ax.set_ylabel('% of Rides (within rider type)')  
ax.legend()  
ax.set_ylim(0, 75)  
  
plt.tight_layout()  
plt.savefig('02_duration_buckets.png', dpi=150, bbox_inches='tight')  
plt.show()  
print("✔ Saved: 02_duration_buckets.png")
```



🔍 Step 8: Hourly Ride Pattern Analysis

Examines usage trends across 24-hour cycle.

— Section 5: Hourly Pattern —

```

hourly = df.groupby(['member_casual', 'hour_of_day'])\
    .size().reset_index(name='rides')

fig, ax = plt.subplots(figsize=(13, 6))

for rtype, color in [('casual', COLORS['casual']),
                    ('member', COLORS['member'])]:
    data = hourly[hourly['member_casual'] == rtype]
    ax.plot(data['hour_of_day'], data['rides'],
            color=color, linewidth=2.5, marker='o',
            markersize=5, label=rtype.capitalize())
    ax.fill_between(data['hour_of_day'], data['rides'],
                    alpha=0.12, color=color)

# Annotate commute peaks
for hour, label, offset in [(8, 'Morning\nCommute', 15),
                            (17, 'Evening\nCommute', 15)]:
    m_val = hourly[(hourly['member_casual']=='member') &
                    (hourly['hour_of_day']==hour)]['rides'].values
    if len(m_val):
        ax.annotate(label,
                    xy=(hour, m_val[0]),
                    xytext=(hour, m_val[0] + offset),
                    fontsize=9, ha='center', color=COLORS['member'],
                    arrowprops=dict(arrowstyle='->',
                                    color=COLORS['member'], lw=1.2))

ax.set_title('Hourly Ride Pattern: Member vs Casual')
ax.set_xlabel('Hour of Day')
ax.set_ylabel('Number of Rides (sample)')
ax.set_xticks(range(0, 24))

```

```

ax.set_xticklabels([f'{h:02d}:00' for h in range(24)],
                    rotation=45, fontsize=8)

ax.legend()

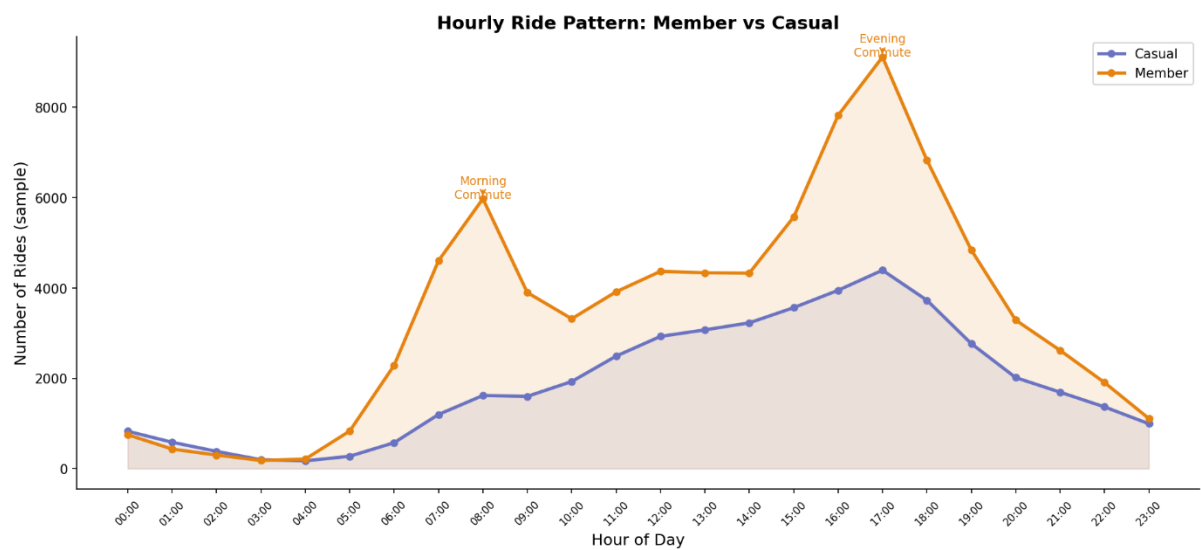
plt.tight_layout()

plt.savefig('03_hourly_pattern.png', dpi=150, bbox_inches='tight')

plt.show()

print("✔ Saved: 03_hourly_pattern.png")

```



Step 9: Weekly Usage Patterns

Comparison of weekday vs weekend behavior.

— Section 6: Day of Week —

```

day_order = ['Monday', 'Tuesday', 'Wednesday',
              'Thursday', 'Friday', 'Saturday', 'Sunday']

daily = df.groupby(['member_casual', 'day_of_week'])\
        .size().reset_index(name='rides')

```

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

fig.suptitle('Weekly Ride Patterns', fontsize=15,

```

```

        fontweight='bold')

# Plot 1 — Side-by-side bars
ax1 = axes[0]
x = np.arange(len(day_order))
width = 0.38

for i, (rtype, color) in enumerate(
    [('casual', COLORS['casual']), ('member', COLORS['member'])]):
    vals = [daily[(daily['member_casual']==rtype) &
                  (daily['day_of_week']==d)][rides].values
            for d in day_order]
    vals = [v[0] if len(v) else 0 for v in vals]
    ax1.bar(x + (i-0.5)*width, vals, width,
            label=rtype.capitalize(), color=color, alpha=0.88)

ax1.set_xticks(x)
ax1.set_xticklabels([d[:3] for d in day_order])
ax1.set_title('Total Rides by Day')
ax1.set_ylabel('Rides (sample)')
ax1.legend()

# Add weekend shading
ax1.axvspan(4.6, 6.5, alpha=0.07, color='gray',
            label='Weekend')
ax1.text(5.5, ax1.get_ylim()[1]*0.95, 'Weekend',
         ha='center', fontsize=9, color='gray')

# Plot 2 — Avg duration by day
ax2 = axes[1]
dur_daily = df.groupby(['member_casual', 'day_of_week'])\

```

```

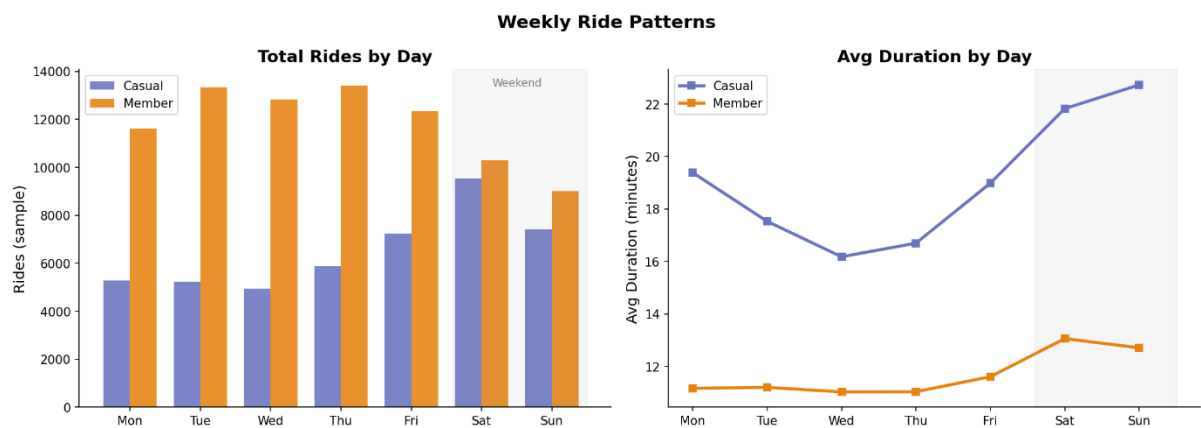
[ride_duration'].mean().reset_index()

for rtype, color in [('casual', COLORS['casual']),
                    ('member', COLORS['member'])]:
    vals = [dur_daily[(dur_daily['member_casual']==rtype) &
                    (dur_daily['day_of_week']==d)][ride_duration'].values
            for d in day_order]
    vals = [v[0] if len(v) else 0 for v in vals]
    ax2.plot(range(len(day_order)), vals, color=color,
            linewidth=2.5, marker='s', markersize=6,
            label=rtype.capitalize())

ax2.set_xticks(range(len(day_order)))
ax2.set_xticklabels([d[:3] for d in day_order])
ax2.set_title('Avg Duration by Day')
ax2.set_ylabel('Avg Duration (minutes)')
ax2.legend()
ax2.axvspan(4.6, 6.5, alpha=0.07, color='gray')

plt.tight_layout()
plt.savefig('04_weekly_pattern.png', dpi=150, bbox_inches='tight')
plt.show()
print("✔ Saved: 04_weekly_pattern.png")

```



🚲 Step 10: Bike Type Usage Analysis

Examines preference and duration across bike types.

```
# — Section 7: Bike Type Analysis —
```

```
bike = df.groupby(['member_casual', 'rideable_type'])\  
    .agg(rides=('ride_duration', 'count'),  
         avg_dur=('ride_duration', 'mean'))\  
    .round(2).reset_index()
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))  
fig.suptitle('Bike Type Usage Analysis', fontsize=15,  
             fontweight='bold')
```

```
bike_types = bike['rideable_type'].unique()  
x = np.arange(len(bike_types))  
width = 0.38
```

```
# Plot 1 — Ride count by bike type
```

```
ax1 = axes[0]  
for i, (rtype, color) in enumerate(  
    [('casual', COLORS['casual']), ('member', COLORS['member'])]):  
    vals = [bike[(bike['member_casual']==rtype) &  
                (bike['rideable_type']==b)]['rides'].values  
            for b in bike_types]  
    vals = [v[0] if len(v) else 0 for v in vals]  
    ax1.bar(x + (i-0.5)*width, vals, width,  
            label=rtype.capitalize(), color=color, alpha=0.88)  
  
ax1.set_xticks(x)  
ax1.set_xticklabels([b.replace('_', '\n') for b in bike_types])
```



```

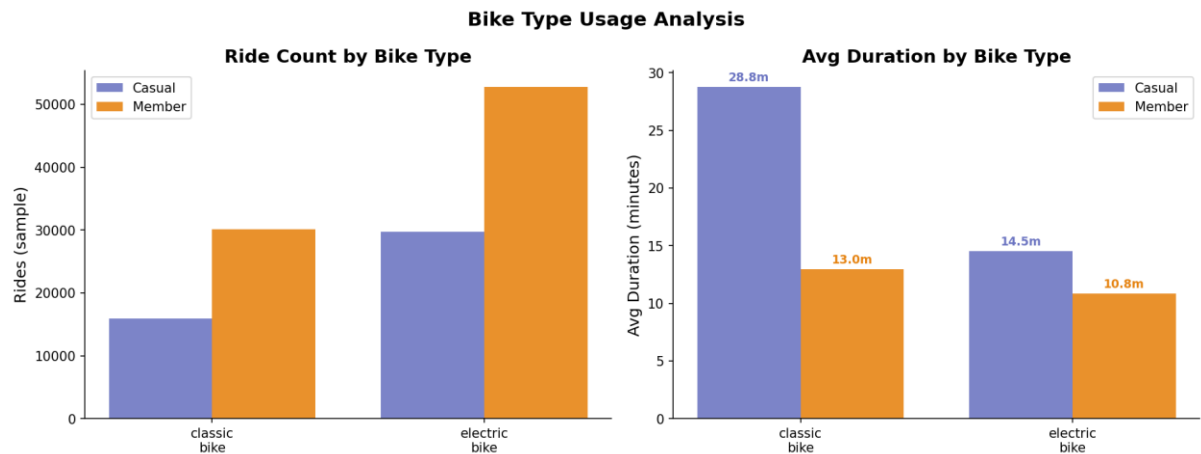
ax1.set_title('Ride Count by Bike Type')
ax1.set_ylabel('Rides (sample)')
ax1.legend()

# Plot 2 — Avg duration by bike type
ax2 = axes[1]
for i, (rtype, color) in enumerate(
    [('casual', COLORS['casual']), ('member', COLORS['member'])]):
    vals = [bike[(bike['member_casual']==rtype) &
        (bike['rideable_type']==b)]['avg_dur'].values
        for b in bike_types]
    vals = [v[0] if len(v) else 0 for v in vals]
    bars = ax2.bar(x + (i-0.5)*width, vals, width,
        label=rtype.capitalize(), color=color, alpha=0.88)
    for bar, val in zip(bars, vals):
        ax2.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 0.3,
            f'{val:.1f}m', ha='center', va='bottom',
            fontsize=9, fontweight='bold', color=color)

ax2.set_xticks(x)
ax2.set_xticklabels([b.replace('_', '\n') for b in bike_types])
ax2.set_title('Avg Duration by Bike Type')
ax2.set_ylabel('Avg Duration (minutes)')
ax2.legend()

plt.tight_layout()
plt.savefig('05_bike_type.png', dpi=150, bbox_inches='tight')
plt.show()
print("✔ Saved: 05_bike_type.png")

```



🔄 Step 11: Round Trip vs One-Way Analysis

— Section 8: Round Trip Analysis —

```
rt = df.groupby(['member_casual', 'trip_type'])\
    .size().reset_index(name='rides')

rt['pct'] = rt.groupby('member_casual')['rides']\
    .transform(lambda x: x/x.sum()*100)

fig, ax = plt.subplots(figsize=(9, 5))

trip_types = rt['trip_type'].unique()
x = np.arange(len(trip_types))
width = 0.35

for i, (rtype, color) in enumerate(
    [('casual', COLORS['casual']), ('member', COLORS['member'])]):
    vals = [rt[(rt['member_casual']==rtype) &
               (rt['trip_type']==t)]['pct'].values
            for t in trip_types]
    vals = [v[0] if len(v) else 0 for v in vals]
    bars = ax.bar(x + (i-0.5)*width, vals, width,
                  label=rtype.capitalize(), color=color, alpha=0.88)
```

```

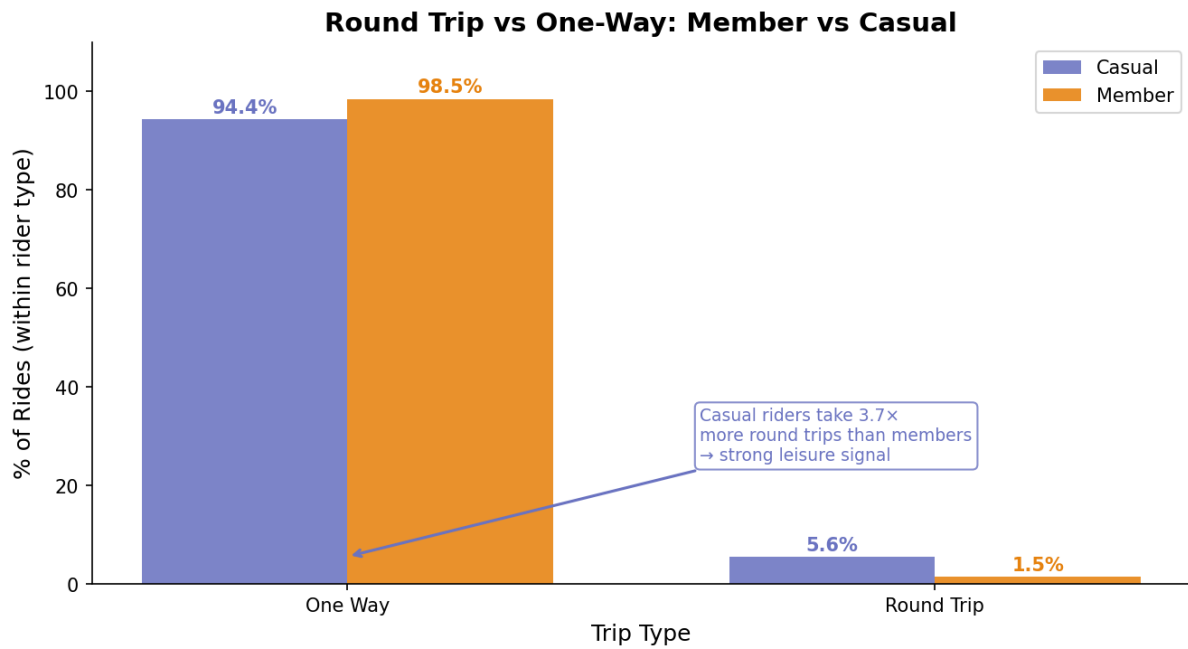
for bar, val in zip(bars, vals):
    ax.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 0.5,
            f'{val:.1f}%', ha='center', va='bottom',
            fontsize=10, fontweight='bold', color=color)

ax.set_xticks(x)
ax.set_xticklabels(trip_types)
ax.set_title('Round Trip vs One-Way: Member vs Casual')
ax.set_xlabel('Trip Type')
ax.set_ylabel('% of Rides (within rider type)')
ax.legend()
ax.set_ylim(0, 110)

# Key insight annotation
ax.annotate('Casual riders take 3.7×\nmore round trips than members\n→ strong leisure signal',
           xy=(0, rt[(rt['member_casual']=='casual') &
                    (rt['trip_type']=='Round Trip')]['pct'].values[0]),
           xytext=(0.6, 25),
           fontsize=9, color=COLORS['casual'],
           arrowprops=dict(arrowstyle='->',
                           color=COLORS['casual'], lw=1.5),
           bbox=dict(boxstyle='round,pad=0.3',
                     facecolor='white', edgecolor=COLORS['casual'],
                     alpha=0.8))

plt.tight_layout()
plt.savefig('06_round_trip.png', dpi=150, bbox_inches='tight')
plt.show()
print("✔ Saved: 06_round_trip.png")

```



🔗 Step 12: Correlation Analysis

Evaluates relationships between key numeric variables.

— Section 9: Correlation Analysis —

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle('Correlation Analysis by Rider Type',
             fontsize=15, fontweight='bold')
```

```
numeric_cols = ['ride_duration', 'distance_miles', 'hour_of_day']
```

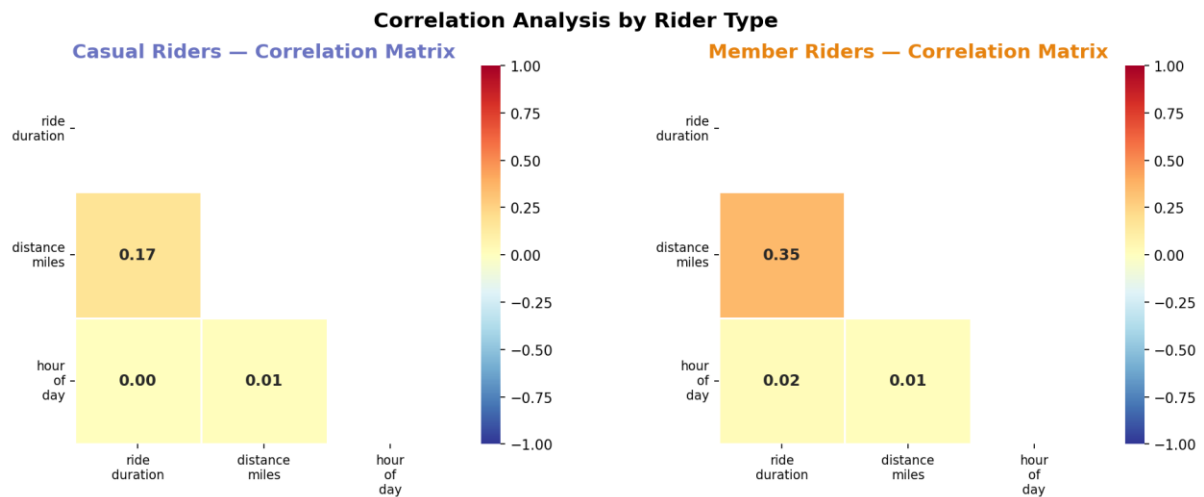
```
for i, (rtype, color, ax) in enumerate(zip(
    ['casual', 'member'],
    [COLORS['casual'], COLORS['member']],
    axes)):
    subset = df[df['member_casual'] == rtype][numeric_cols]
    corr = subset.corr()
```

```
mask = np.triu(np.ones_like(corr, dtype=bool))

sns.heatmap(corr, ax=ax, annot=True, fmt='.2f',
            cmap='RdYlBu_r', center=0,
            mask=mask,
            square=True, linewidths=0.5,
            annot_kws={'size': 11, 'weight': 'bold'},
            vmin=-1, vmax=1)

ax.set_title(f'{rtype.capitalize()} Riders — Correlation Matrix',
            color=color)
ax.set_xticklabels(
    [c.replace('_', '\n') for c in numeric_cols],
    rotation=0, fontsize=9)
ax.set_yticklabels(
    [c.replace('_', '\n') for c in numeric_cols],
    rotation=0, fontsize=9)

plt.tight_layout()
plt.savefig('07_correlation_heatmap.png', dpi=150,
            bbox_inches='tight')
plt.show()
print("✔ Saved: 07_correlation_heatmap.png")
```



🛠️ Step 13: Seasonal Trend Analysis

— Section 10: Season Analysis —

```
season_order = ['Spring', 'Summer', 'Autumn', 'Winter']
```

```
season_colors = {
```

```
    'Spring': '#4CAF50', 'Summer': '#FF9800',
```

```
    'Autumn': '#FF5722', 'Winter': '#2196F3'
```

```
}
```

```
seasonal = df.groupby(['member_casual', 'season'])\
```

```
    .agg(rides=('ride_duration', 'count'),
```

```
         avg_dur=('ride_duration', 'mean'))\
```

```
    .reset_index()
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
fig.suptitle('Seasonal Ride Patterns', fontsize=15,
```

```
            fontweight='bold')
```

```
# Plot 1 — Rides by season
```

```
ax1 = axes[0]
```

```
x = np.arange(len(season_order))
```

```
width = 0.38
```

```
for i, (rtype, color) in enumerate(
    [('casual', COLORS['casual']), ('member', COLORS['member'])]):
    vals = [seasonal[(seasonal['member_casual']==rtype) &
                    (seasonal['season']==s)]['rides'].values
            for s in season_order]
    vals = [v[0] if len(v) else 0 for v in vals]
    ax1.bar(x + (i-0.5)*width, vals, width,
            label=rtype.capitalize(), color=color, alpha=0.88)
```

```
ax1.set_xticks(x)
ax1.set_xticklabels(season_order)
ax1.set_title('Total Rides by Season')
ax1.set_ylabel('Rides (sample)')
ax1.legend()
```

```
# Plot 2 — Avg duration by season
```

```
ax2 = axes[1]
for rtype, color in [('casual', COLORS['casual']),
                    ('member', COLORS['member'])]:
    vals = [seasonal[(seasonal['member_casual']==rtype) &
                    (seasonal['season']==s)]['avg_dur'].values
            for s in season_order]
    vals = [v[0] if len(v) else 0 for v in vals]
    ax2.plot(range(len(season_order)), vals,
            color=color, linewidth=2.5, marker='D',
            markersize=8, label=rtype.capitalize())
```

```
ax2.set_xticks(range(len(season_order)))
ax2.set_xticklabels(season_order)
```

```

ax2.set_title('Avg Duration by Season')

ax2.set_ylabel('Avg Duration (minutes)')

ax2.legend()

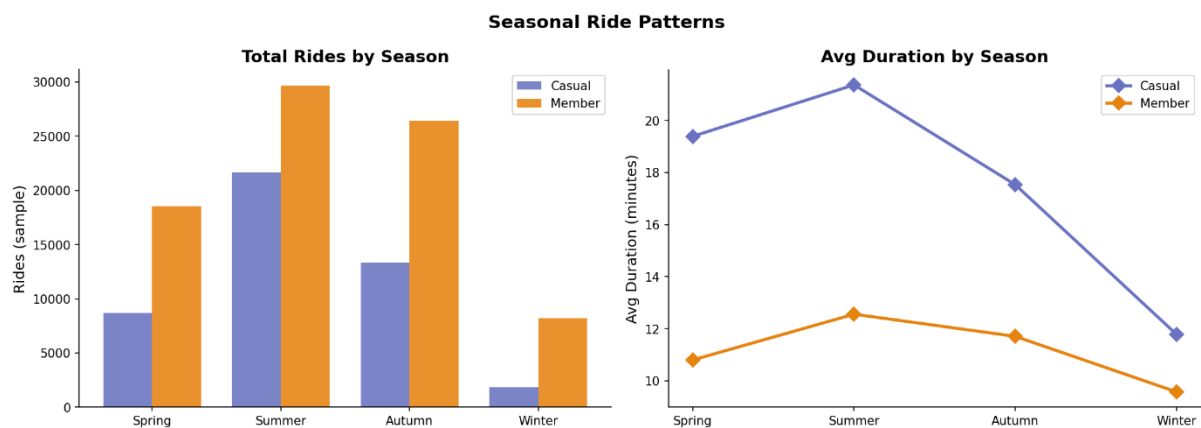
plt.tight_layout()

plt.savefig('08_seasonal_analysis.png', dpi=150,
           bbox_inches='tight')

plt.show()

print("✔ Saved: 08_seasonal_analysis.png")

```



📄 Step 14: Final Summary Report

Automated reporting of key insights.

— Section 11: Summary Statistics Report —

```

print("=" * 60)

print(" CYCLISTIC 2025 — PYTHON ANALYSIS SUMMARY REPORT")

print("=" * 60)

```

```

m = df[df['member_casual']=='member']
c = df[df['member_casual']=='casual']

```

```

print(f"""
DATASET

```


Total sample rides : {len(df):,}

Members : {len(m):,} ({len(m)/len(df)*100:.1f}%)

Casual riders : {len(c):,} ({len(c)/len(df)*100:.1f}%)

RIDE DURATION

Member — Mean : {m['ride_duration'].mean():.2f} min

Member — Median : {m['ride_duration'].median():.0f} min

Casual — Mean : {c['ride_duration'].mean():.2f} min

Casual — Median : {c['ride_duration'].median():.0f} min

Duration gap : Casual rides {c['ride_duration'].mean()/m['ride_duration'].mean():.1f}x longer (mean)

STATISTICAL SIGNIFICANCE

Welch's T-test p-value : {p_value:.2e}

Cohen's d (effect size): {d:.4f} ({magnitude} effect)

Mann-Whitney p-value : {p_mw:.2e}

Conclusion: Difference in ride duration is STATISTICALLY

SIGNIFICANT ($p < 0.001$) with a {magnitude} effect size

DISTANCE

Member — Mean : {m['distance_miles'].mean():.2f} miles

Casual — Mean : {c['distance_miles'].mean():.2f} miles

Distance gap : Minimal — behavior differs in TIME not DISTANCE

ROUND TRIPS (leisure signal)

Casual round trip rate : {(c['trip_type']=='Round Trip').mean()*100:.1f}%

Member round trip rate : {(m['trip_type']=='Round Trip').mean()*100:.1f}%

TOP INSIGHTS

1. Casual riders take {c['ride_duration'].mean()/m['ride_duration'].mean():.1f}x longer rides

→ suggests leisure, not commuting

2. Members have clear 8am & 5pm peaks

→ confirms commuter behavior pattern

3. Casual riders prefer weekends; members prefer weekdays

→ different use cases, same product

4. Casual classic bike avg: {c[c['rideable_type']=='classic_bike']['ride_duration'].mean():.1f} min

vs member classic bike: {m[m['rideable_type']=='classic_bike']['ride_duration'].mean():.1f} min

→ largest gap is on classic bikes

FILES SAVED

01_ride_duration_distribution.png

02_duration_buckets.png

03_hourly_pattern.png

04_weekly_pattern.png

05_bike_type.png

06_round_trip.png

07_correlation_heatmap.png

08_seasonal_analysis.png

```
""")
```

```
print("=" * 60)
```

```
print(" ✔ ANALYSIS COMPLETE")
```

```
print("=" * 60)
```

```
=====
```

CYCLISTIC 2025 — PYTHON ANALYSIS SUMMARY REPORT

```
=====
```

DATASET

Total sample rides : 128,395

Members : 82,838 (64.5%)

Casual riders : 45,557 (35.5%)

RIDE DURATION

Member — Mean : 11.60 min

Member — Median : 8 min

Casual — Mean : 19.47 min

Casual — Median : 11 min

Duration gap : Casual rides 1.7x longer (mean)

STATISTICAL SIGNIFICANCE

Welch's T-test p-value : 0.00e+00

Cohen's d (effect size): -0.2577 (small effect)

Mann-Whitney p-value : 0.00e+00

Conclusion: Difference in ride duration is STATISTICALLY
SIGNIFICANT ($p < 0.001$) with a small effect size

DISTANCE

Member — Mean : 1.41 miles

Casual — Mean : 1.40 miles

Distance gap : Minimal — behavior differs in TIME not DISTANCE

ROUND TRIPS (leisure signal)

Casual round trip rate : 5.6%

Member round trip rate : 1.5%

TOP INSIGHTS

1. Casual riders take 1.7x longer rides
→ suggests leisure, not commuting
2. Members have clear 8am & 5pm peaks
→ confirms commuter behavior pattern
3. Casual riders prefer weekends; members prefer weekdays
→ different use cases, same product
4. Casual classic bike avg: 28.8 min
vs member classic bike: 13.0 min

→ largest gap is on classic bikes

FILES SAVED

01_ride_duration_distribution.png

02_duration_buckets.png

03_hourly_pattern.png

04_weekly_pattern.png

05_bike_type.png

06_round_trip.png

07_correlation_heatmap.png

08_seasonal_analysis.png

=====

✓ ANALYSIS COMPLETE

=====